ASSIGNMENT 1

1. Explain the difference between frontend, backend, and full-stack development with suitable real-world examples.

Frontend Development	Backend Development	Full-Stack Development
Focuses on the **user interface (UI)** and everything users see and interact with.	Focuses on the **server, database, and application logic** that work behind the scenes.	Combines both **frontend and backend development** skills.
Uses technologies like **HTML, CSS, JavaScript**, and frameworks such as React, Angular, or Vue.js.	Uses languages like **Python, Java, PHP, Node.js, C#**, and databases like MySQL or MongoDB.	Uses both frontend and backend technologies to build complete web applications.
Responsible for creating responsive layouts, buttons, forms, menus, and animations.	Responsible for authentication, data storage, business logic, and API development.	Handles the complete development process from designing the interface to managing the database.

Real-World Example: Online Shopping Website (e.g., Amazon)

1. Frontend Development

- Designs the homepage, product pages, shopping cart, and checkout screens.
- Ensures buttons, images, menus, and search bars work properly.
- Example: When you click the **"Add to Cart"** button, the frontend captures your action.

2. Backend Development

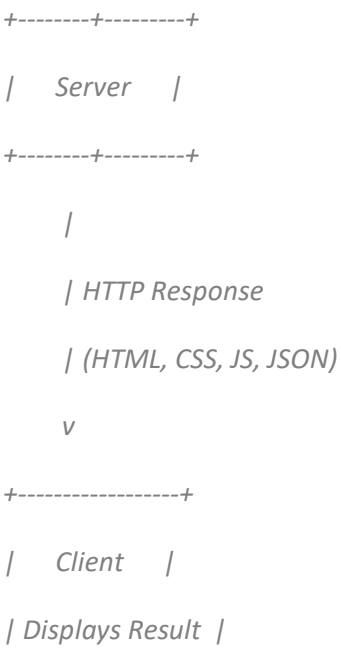
- Processes the request from the frontend.
- Checks whether the product is in stock.
- Stores the cart information in the database.
- Calculates the total price and manages payment processing.
- Example: After clicking **"Add to Cart"**, the backend updates your shopping cart in the database.

3. Full-Stack Development

- A full-stack developer builds both the shopping website's interface and its server-side functionality.
- They create the product pages (frontend) and also develop the database, APIs, and payment logic (backend).

2. Create a simple diagram showing how the client-server model works in web architecture.


Client
(Web Browser)
Chrome, Firefox
HTTP Request
(URL, Form Data)
Server
(Web Application)
Apache, Nginx,
Node.js, Java
Database Query
Database
MySQL, MongoDB
Data Returned



Working Process

1. **Client (Browser)** sends a request to the server.
2. **Server** receives the request and processes it.
3. If data is needed, the server queries the **Database**.
4. The database returns the required data to the server.
5. The server prepares a response (web page or data).

6. The response is sent back to the client.
7. The browser displays the result to the user.

Real-World Example: Login to Instagram

- User enters username and password in the browser (**Client**).
- Browser sends login request to Instagram's **Server**.
- Server checks credentials in the **Database**.
- If valid, the server sends a success response.
- The browser displays the user's Instagram feed.

Flow:
Client → Server → Database →

Server → Client.

3. Describe how a browser requests and displays a web page from a web server.

When a user enters a website address (URL) in a browser, the browser and server communicate to fetch and display the web page.

Step-by-Step Process

1. **User Enters URL**
   - The user types a URL such as `www.google.com` into the browser.
2. **DNS Lookup**
   - The browser contacts a DNS (Domain Name System) server to find the IP address associated with the domain name.
3. **Browser Sends HTTP Request**
   - Using the IP address, the browser sends an HTTP/HTTPS request to the web server asking for the desired web page.
4. **Web Server Processes Request**
   - The web server receives the request and locates the required files or data.
   - If needed, it interacts with databases and application logic.
5. **Server Sends HTTP Response**
   - The server returns an HTTP response containing HTML, CSS, JavaScript, images, and other resources.
6. **Browser Receives Response**
   - The browser downloads the received files.
7. **Browser Renders the Page**
   - HTML is parsed to create the page structure.
   - CSS is applied for styling.
   - JavaScript is executed to add interactivity.
8. **Web Page Displayed**
   - The fully rendered page is shown to the user.

User

|

v

Browser

|

| HTTP/HTTPS Request

v

Web Server

|

| HTTP Response (HTML, CSS, JS)

v

Browser

|

v

Web Page Displayed

4. Identify and list the tools required to set up a web development environment. Explain the purpose of each.

Tool	Purpose
Text Editor / IDE (VS Code, IntelliJ IDEA, Sublime Text)	Used to write and edit source code efficiently. Provides features like syntax highlighting, auto-completion, and debugging.
Web Browser (Chrome, Firefox, Edge)	Used to run and test web pages. Browser Developer Tools help inspect HTML, CSS, JavaScript, and debug issues.
Version Control System (Git)	Tracks code changes, allows collaboration, and maintains project history.
Git Repository Hosting (GitHub, GitLab)	Stores code online and enables team collaboration.
Programming Languages	HTML for structure, CSS for styling, JavaScript for interactivity, and backend languages such as Java, Python, or PHP.
Package Manager (npm)	Installs and manages libraries and dependencies required by the project.
Runtime Environment (Node.js)	Executes JavaScript outside the browser and supports backend development.
Local Web Server (Apache, Nginx, Node.js Server)	Hosts and serves web applications during development and testing.
Database System (MySQL, MongoDB, PostgreSQL)	Stores and manages application data.
Browser Developer Tools	Used for debugging, inspecting elements, monitoring network requests, and testing responsiveness.
Frameworks/Libraries (React, Angular, Bootstrap, Express.js)	Speed up development by providing reusable components and functionality.

5. Explain what a web server is and give examples of commonly used servers.

A **web server** is software (and sometimes the computer running it) that stores, processes, and delivers web pages and other web content to users over the Internet using the **HTTP/HTTPS** protocol.

When a user enters a website URL in a browser, the browser sends a request to the web server. The web server processes the request and sends back the requested web page, images, videos, or other resources.

How a Web Server Works

1. User enters a website URL in a browser.
2. Browser sends an HTTP/HTTPS request to the web server.
3. The web server receives and processes the request.
4. If required, it retrieves data from a database.
5. The server sends an HTTP response back to the browser.
6. The browser displays the web page.

User Browser

|

| HTTP Request

v

+-------------+

| Web Server |

+-------------+

|

| HTTP Response

v

User Browser

Web Server	Description
Apache HTTP Server	One of the most popular open-source web servers. Supports multiple operating systems and modules.
Nginx	High-performance web server widely used for handling large amounts of traffic and as a reverse proxy.
Microsoft IIS	Internet Information Services (IIS) is Microsoft's web server for Windows-

Web Server	Description
	based systems.

7. Define the roles of a frontend developer, backend developer, and database administrator in a project.

In a web development project, different professionals handle different responsibilities to ensure the application works efficiently.

Role	Responsibilities
Frontend Developer	Develops the user interface (UI) and user experience (UX). Creates web pages using HTML, CSS, JavaScript, and frameworks like React or Angular.
Backend Developer	Develops server-side logic, APIs, authentication, and application functionality. Uses languages such as Java, Python, PHP, or Node.js.
Database Administrator (DBA)	Designs, manages, secures, and maintains databases. Ensures data integrity, backup, recovery, and performance optimization.

1. Frontend Developer

A frontend developer is responsible for everything the user sees and interacts with on a website.

Main Duties

- Designing web page layouts.
- Creating responsive user interfaces.
- Implementing navigation menus, forms, and buttons.
- Improving user experience and accessibility.
- Integrating frontend with backend APIs.

Technologies Used

- HTML
- CSS
- JavaScript
- React, Angular, Vue.js

Example

On an online shopping website, the frontend developer creates:

- Product pages
- Search bar

- Shopping cart interface
- Checkout page

---

2. Backend Developer

A backend developer handles the server-side operations and business logic.

Main Duties

- Creating APIs and server logic.
- Managing user authentication and authorization.
- Processing requests from the frontend.
- Connecting applications with databases.
- Ensuring security and performance.

Technologies Used

- Java (Spring Boot)
- Python (Django, Flask)
- Node.js
- PHP
- C#

Example

On an online shopping website, the backend developer:

- Verifies login credentials.
- Processes orders and payments.
- Updates inventory.
- Retrieves product information from the database.

---

3. Database Administrator (DBA)

A Database Administrator manages the database system that stores application data.

Main Duties

- Designing database schemas and tables.
- Managing data storage and retrieval.
- Performing backups and recovery.
- Monitoring database performance.

- Implementing security and access controls.

Technologies Used

- MySQL
- PostgreSQL
- Oracle Database
- MongoDB
- SQL Server

Example

On an online shopping website, the DBA manages:

- Customer records
- Product details
- Order history
- Payment information

8. Install VS Code and configure it for HTML, CSS, and JavaScript development. Take a screenshot of the setup.

7. Install VS Code and Configure It for HTML, CSS, and JavaScript Development

Step 1: Install VS Code

1. Visit: Visual Studio Code
2. Download the Windows version.
3. Run the installer.
4. Check:
    - Add "Open with Code" action
    - Add to PATH
    - Register Code as editor
5. Click **Install** and then **Finish**.

Step 2: Create a Project Folder

Create a folder named:

```
WebDevelopment
```

Open it in VS Code:

- File → Open Folder → WebDevelopment

Step 3: Install Useful Extensions

Click the **Extensions** icon (Ctrl+Shift+X) and install:

1. **Live Server**
    - Runs HTML pages in a browser automatically.
2. **Prettier**
    - Formats HTML, CSS, and JavaScript code.
3. **HTML CSS Support**
    - Provides CSS suggestions in HTML files.
4. **JavaScript (ES6) Code Snippets**
    - Useful JavaScript shortcuts.

Step 4: Create Files

Create:

```
index.html
style.css
script.js
```

Step 5: Add Sample Code

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>My First Web Page</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <h1>Hello World</h1>
    <button onclick="showMessage()">Click Me</button>

    <script src="script.js"></script>
</body>
</html>
```

style.css

```
body{
    text-align:center;
    font-family:Arial;
}

h1{
    color:blue;
}
```

script.js

```
function showMessage(){
    alert("Welcome to Web Development!");
}
```

Step 6: Run the Project

1. Right-click `index.html`
2. Click **Open with Live Server**
3. Browser opens automatically.

Screenshot Required for Practical File

Take a screenshot showing:

- VS Code open
- `index.html`, `style.css`, and `script.js` files visible
- Extensions installed (Live Server, Prettier)
- Browser displaying the page "Hello World"

Sample Screenshot Layout

```
+-------------------------------------------------+
| VS CODE                                         |
|-------------------------------------------------|
| index.html | style.css | script.js              |
|                                                 |
| <h1>Hello World</h1>                            |
|                                                 |
+-------------------------------------------------+

Browser Output:
-----------------------------------
Hello World

[ Click Me ]
-----------------------------------
```

8. Explain the difference between static and dynamic websites. Provide an example of each.

A website can be classified as **static** or **dynamic** based on how its content is generated and displayed to users.

Static Website	Dynamic Website
Content remains the same for all users unless manually updated.	Content can change based on user interaction, database data, or other conditions.
Built mainly using HTML, CSS, and basic	Uses HTML, CSS, JavaScript along with backend technologies

Static Website	Dynamic Website
JavaScript.	such as PHP, Java, Python, or Node.js.
Does not require a database.	Usually connected to a database.
Faster and easier to develop.	More complex but provides interactive features.
Suitable for small websites and portfolios.	Suitable for large websites, e-commerce sites, and web applications.

1. Static Website

A **static website** consists of fixed web pages. Every user sees the same content unless the developer manually edits the files.

Characteristics

- Simple and fast.
- No server-side processing.
- Content is pre-written and stored in HTML files.

Example

A personal portfolio website containing:

- Home page
- About page
- Contact page

Every visitor sees the same information.

Example Websites:

- Personal portfolio
- Company brochure website
- College information page

---

2. Dynamic Website

A **dynamic website** generates content at runtime based on user requests, database information, or business logic.

Characteristics

- Interactive and personalized.
- Uses databases.
- Content changes automatically.

Example

An online shopping website like **Amazon**:

- Different users see different recommendations.
- Product information is fetched from a database.
- Cart and order history are unique for each user.

Other Examples:

- Facebook
- Instagram
- YouTube

Simple Comparison Diagram

```
STATIC WEBSITE
User ---> HTML Page ---> Same Content

DYNAMIC WEBSITE
User ---> Server ---> Database
               |
               v
        Customized Content
```

9. Research and list five web browsers. Explain how rendering engines differ between them.

A **web browser** is software used to access and display web pages. Browsers use **rendering engines** to interpret HTML, CSS, and JavaScript and display the content on the screen.

Common Web Browsers

Browser	Developer	Rendering Engine
Google Chrome	Google	Blink
Mozilla Firefox	Mozilla	Gecko

Browser	Developer	Rendering Engine
Microsoft Edge	Microsoft	Blink
Safari	Apple	WebKit
Opera	Opera Software	Blink

What is a Rendering Engine?

A **rendering engine** is the component of a browser that:

- Reads HTML and CSS.
- Builds the page structure.
- Applies styles.
- Displays text, images, and layouts on the screen.

Different rendering engines may display the same webpage slightly differently.

Major Rendering Engines

1. Blink (Chrome, Edge, Opera)

- Developed by Google.
- Derived from WebKit.
- Optimized for speed and performance.
- Supports modern web standards quickly.

Browsers Using Blink:

- Google Chrome
- Microsoft Edge
- Opera

2. Gecko (Firefox)

- Developed by Mozilla.
- Open-source rendering engine.
- Focuses on standards compliance and user privacy.
- Uses a different rendering architecture from Blink.

Browser Using Gecko:

- Mozilla Firefox

3. WebKit (Safari)

- Developed by Apple.
- Originally used by Chrome before Blink was created.
- Optimized for Apple devices and ecosystems.

Browser Using WebKit:

- Safari

Comparison of Rendering Engines

Feature	Blink	Gecko	WebKit
Developed By	Google	Mozilla	Apple
Used In	Chrome, Edge, Opera	Firefox	Safari
Performance	Very Fast	Fast	Fast
Open Source	Yes	Yes	Yes
Platform Focus	Cross-platform	Cross-platform	Apple Ecosystem
Web Standards Support	Excellent	Excellent	Excellent

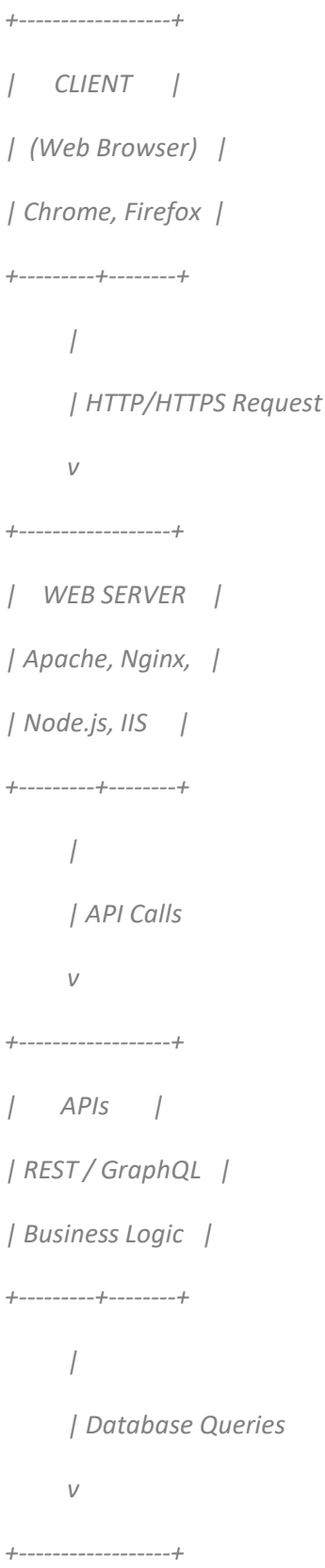
Why Rendering Engines Matter

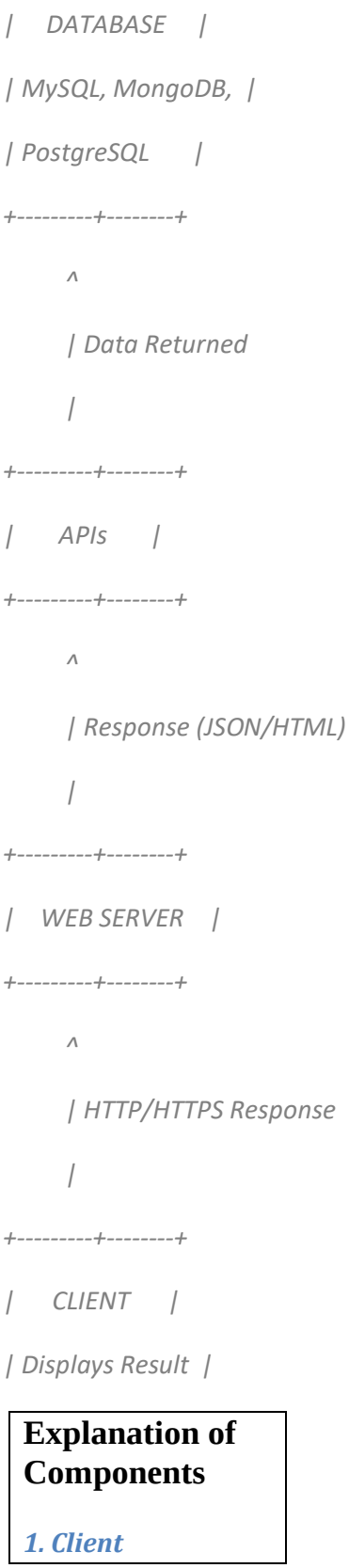
Different rendering engines:

- Interpret web code differently.
- Affect page speed and performance.
- Influence browser compatibility.
- May render layouts and animations slightly differently.

For example, a webpage may look perfect in Chrome (Blink) but require minor adjustments to display correctly in Firefox (Gecko) or Safari (WebKit).

10. Draw a labeled diagram showing the basic web architecture flow — client, server, database, and APIs.


CLIENT
(Web Browser)
Chrome, Firefox
HTTP/HTTPS Request
WEB SERVER
Apache, Nginx,
Node.js, IIS
API Calls
APIs
REST / GraphQL
Business Logic
Database Queries



Explanation of Components

1. Client

- The user's browser or application.
- Sends requests to access web resources.
- Examples: Chrome, Firefox, Edge.

2. Web Server

- Receives requests from clients.
- Processes requests and communicates with APIs.
- Examples: Apache, Nginx, IIS.

3. APIs (Application Programming Interfaces)

- Act as a bridge between the server and database.
- Handle business logic and data processing.
- Common types: REST APIs, GraphQL APIs.

4. Database

- Stores

application data.
- Examples: MySQL, MongoDB, PostgreSQL.

Flow of Data

1. User requests a webpage through a browser (**Client**).
2. The request reaches the **Web Server**.
3. The server calls the required **API**.
4. The API retrieves data from the **Database**.
5. The database returns data to the API.
6. The API sends processed data back to the server.
7. The server sends an HTTP response to the client.
8. The browser displays the webpage.

Example: Online Shopping Website

Client (Browser)
→ User searches for a laptop

Web Server → Receives search request
API → Fetches product data
Database → Stores product details and prices
Response → Product list displayed in the browser

Flow:
Client → Web Server → API → Database → API → Web Server → Client.